DocRAG — Project Report

Oct 23, 2026 · @k Prasanth

Abstract

DocRAG is a retrieval-augmented generation (RAG) system that answers questions about uploaded PDFs and cites the exact file and page behind every claim. It is built from scratch in Python, without frameworks such as LangChain, and runs fully offline: embeddings and answers come from local models served by Ollama, so document content never leaves the machine.

Retrieval combines dense vector search with BM25 keyword search, merges them with Reciprocal Rank Fusion, and re-scores the best candidates with a cross-encoder. On a 30-question test set built from a 41-page project report, this raised top-5 retrieval accuracy from 73% (dense search alone) to **90%**, at about 1.3 s of search time per question on a laptop CPU. The project also includes a Streamlit chat interface, a REST API, a Docker Compose setup, an evaluation suite and 25 automated tests.

Introduction and problem statement

Large language models answer fluently, but they cannot answer reliably about documents they have never seen, such as a company report, a contract or a project thesis. Asked anyway, they tend to invent plausible details (hallucination), and they give no way to check where an answer came from.

Pasting a whole document into the model is not a fix. Long documents exceed the model's context window, the model reads slowly on modest hardware, and details buried in long inputs are often missed. Sending confidential documents to a hosted model service also raises privacy concerns.

Retrieval-augmented generation addresses these problems. The document is split into small passages and indexed once; for each question, only the few most relevant passages are retrieved and given to the model, which is instructed to answer from them alone and cite them. DocRAG implements this pattern end to end, with an emphasis on retrieval quality, verifiable citations, local execution and measurable results.

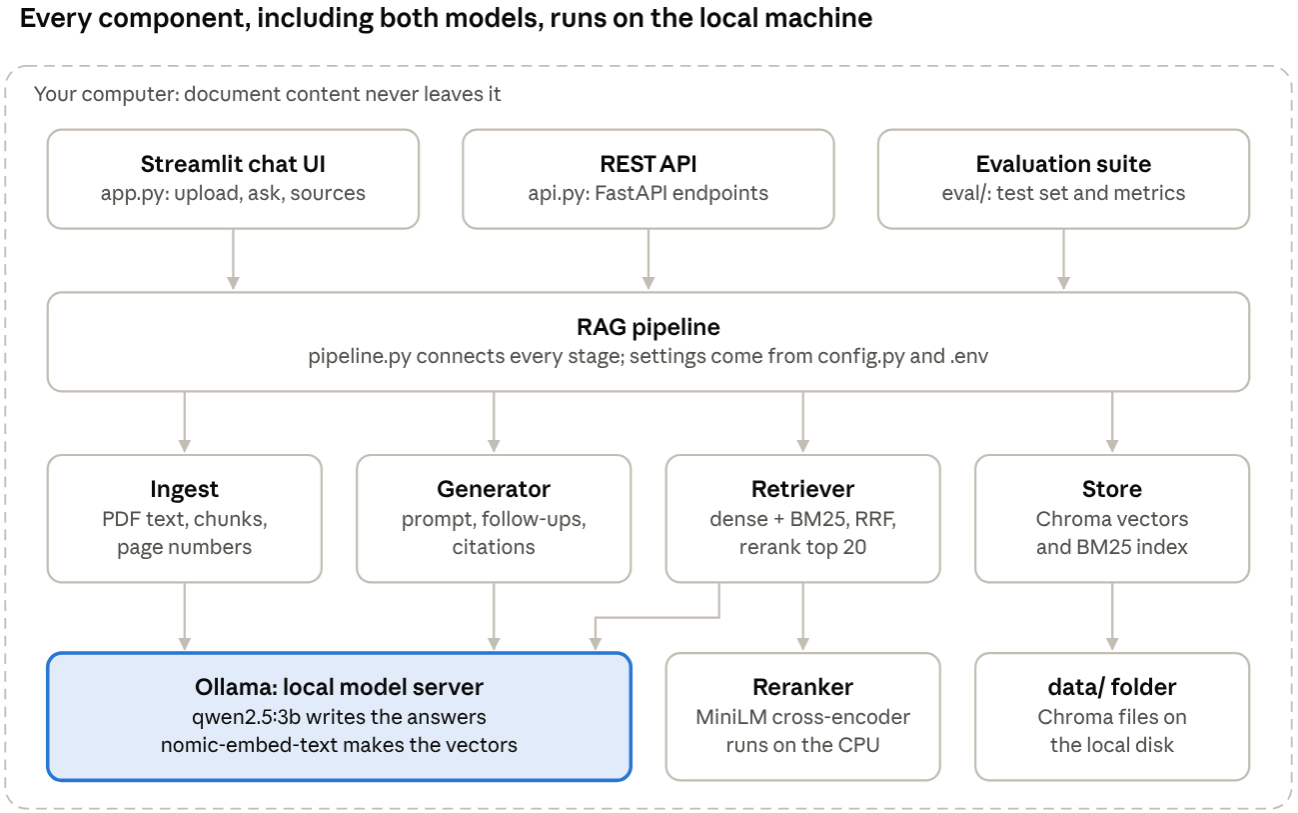
Objectives

1. Answer natural-language questions about any uploaded PDF, using only the document's content.

2. Cite the source file and page for every claim, and say so when the answer is not in the documents.
3. Retrieve accurately for both paraphrased questions and exact terms (names, codes, figures) by combining semantic and keyword search.
4. Run entirely on local hardware, so documents never leave the machine and no API key or usage cost is needed.
5. Support follow-up questions in a conversation.
6. Measure retrieval quality with an evaluation suite, and use the results to choose between design options.
7. Provide a chat interface, a REST API and a one-command Docker setup.

System architecture

DocRAG has four layers: user-facing interfaces, one pipeline that connects every stage, four processing modules, and the local services they rely on.



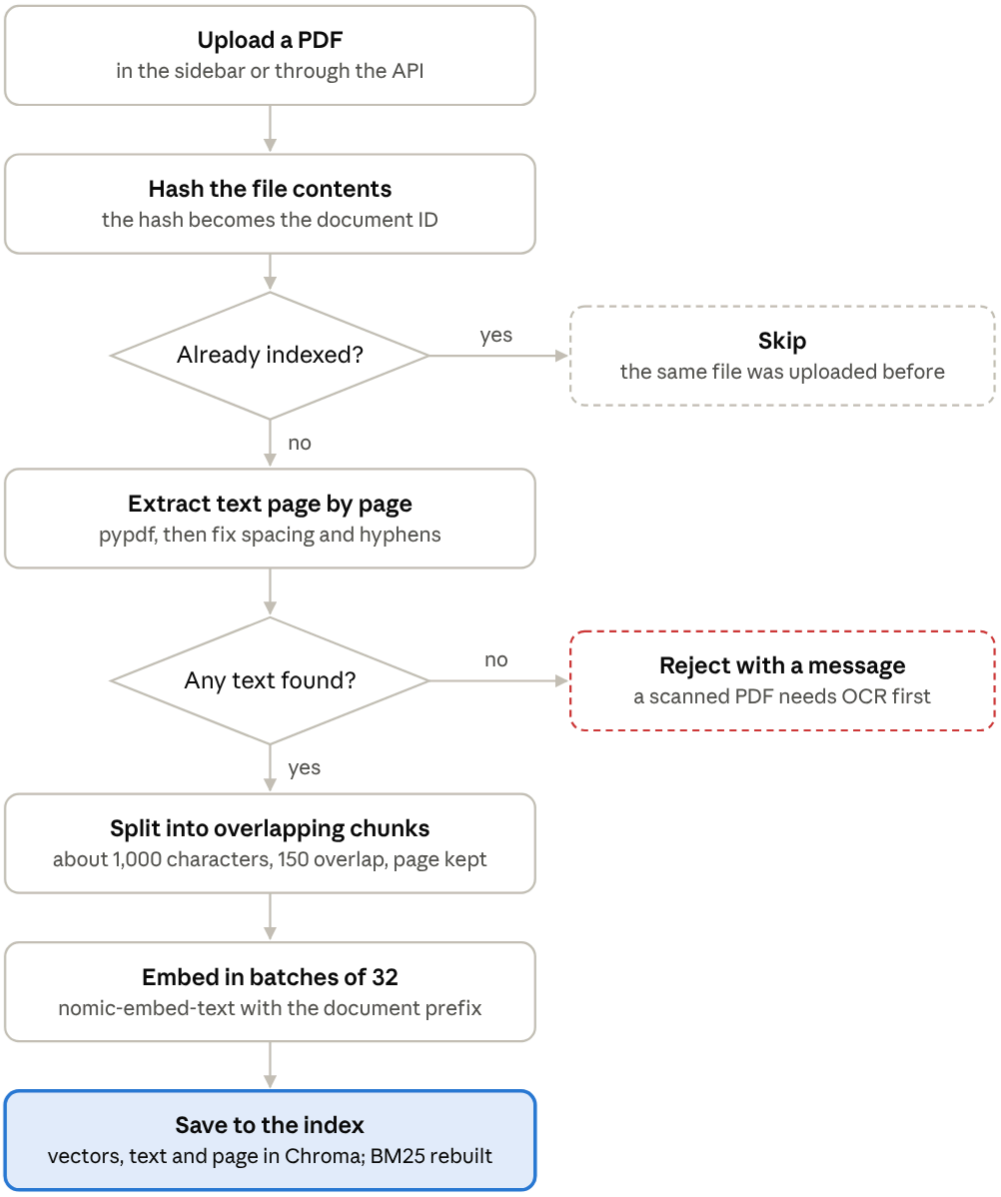
DocRAG architecture · 4 layers, 10 components

The chat UI, the API and the evaluation scripts all call the same pipeline. The ingest module, the generator and the retriever use Ollama for embeddings and answers, the retriever also uses the local reranker, and the store keeps everything in the local data folder, so no stage calls an outside service.

Document ingestion

Each PDF is processed once when it is uploaded; every later question searches the stored result.

Each PDF is read, chunked and embedded once, then searched many times



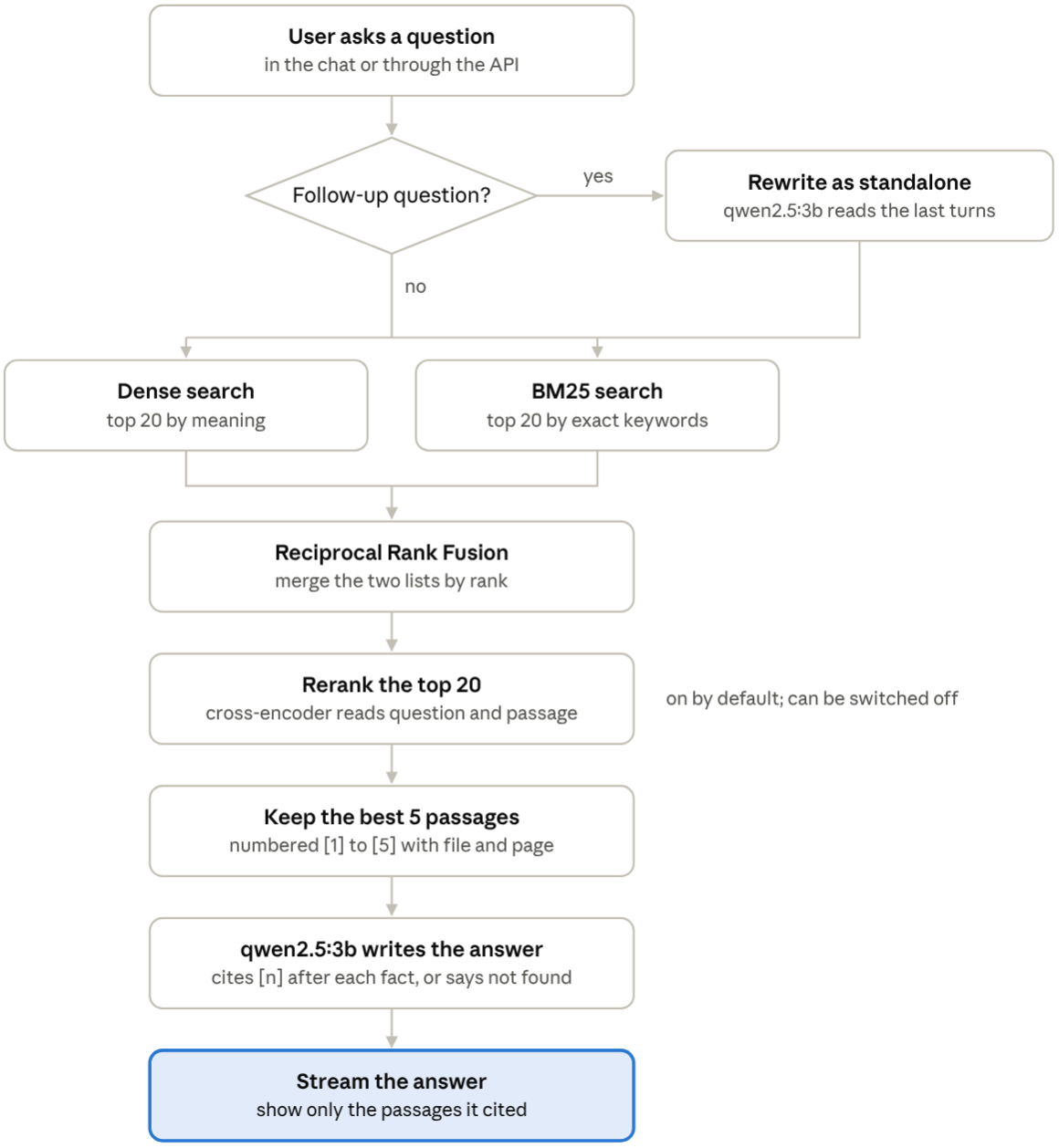
ingestion flow · 6 steps, 2 checks

The content hash makes uploads idempotent, and the text check stops scanned PDFs early with a clear message. Every chunk keeps its file name and page number, which later become the citation.

Question answering

Every question goes through the same path: an optional rewrite, two searches in parallel, fusion, reranking, and a cited answer.

Two searches and a reranker pick the five passages the model reads



question flow · 1 check, 2 parallel searches, 7 steps

The two searches catch different things, meaning and exact terms, and the reranker makes the final choice of five. The model sees only those five passages, so its answer can always be traced to a file and page.

Module design and implementation

The system is written from scratch in Python, with no RAG framework, so that each stage is a small module that can be read, tested and replaced on its own.

Module	Responsibility
`rag/ingest.py`	Reads PDF text page by page, cleans it, splits it into overlapping chunks that keep their file name and page number
`rag/embeddings.py`	Turns chunks and questions into vectors through Ollama, in batches, with model-specific task prefixes
`rag/store.py`	Persistent Chroma vector store, BM25 keyword index and tokenizer, document filtering and deletion, embedding-model guard
`rag/retriever.py`	Dense, BM25 and hybrid retrieval, Reciprocal Rank Fusion, cross-encoder reranking
`rag/generator.py`	Prompt construction, follow-up detection and rewriting, streaming answers, citation parsing
`rag/pipeline.py`	Connects the stages; used by the UI, the API and the evaluation scripts
`rag/config.py`	All settings, read from environment variables or `.env`
`app.py`	Streamlit chat interface with live progress and cited sources
`api.py`	FastAPI REST interface
`eval/`	Test-set generation and retrieval and answer metrics

Chunking

Text is split recursively on paragraph, line, sentence and word boundaries into chunks of about 1,000 characters (roughly 200 to 250 tokens), with a 150-character overlap so an idea is not cut in half. Each chunk keeps its page number, which is what makes page-level citations possible. Each document gets an ID from a hash of its contents, so uploading the same file twice does nothing.

Embeddings

Chunks are embedded with `nomic-embed-text` through Ollama's batch endpoint, 32 at a time. The model is trained with different prefixes for passages and queries, so chunks are embedded as `search_document: …` and questions as `search_query: …`. The index records which model and prefixes built it and refuses to mix vectors from a different model, offering to clear and rebuild instead.

Keyword search and tokenization

BM25 needs words, so DocRAG uses its own word-level tokenizer: lowercase the text, split on anything that is not a letter or digit, and drop common stopwords. A question such as "What does error E-4012 mean?" becomes `error`, `e`, `4012`, `mean`, which is why keyword search finds exact codes and names that vector search can miss.

Fusion and reranking

Dense and BM25 search each return their top 20 passages. Reciprocal Rank Fusion merges the two lists using only each passage's rank, so the two methods' incompatible score scales do not matter:

$$\mathrm{RRF}(d) = \sum_{r \in \{\text{dense},\,\text{BM25}\}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60$$

The top 20 fused passages are then re-scored by a cross-encoder (`ms-marco-MiniLM-L-6-v2`), which reads the question and each passage together rather than comparing separately computed vectors. The best 5 go to the language model.

Follow-up questions

A follow-up such as "Why is that better?" is a poor search query. When a message is short or contains a referring word (it, that, this, they), the model first rewrites it into a standalone question using the last turns of the conversation. Both the search and the answer then use the rewritten question. Self-contained questions skip this step, saving one model call.

Answer generation and citations

The five passages are numbered in the prompt with their file and page. The instructions require a bracketed source number after each fact, ban headings, and require the reply "I couldn't find that in the uploaded documents" when the passages do not contain the answer. A worked example shows the expected style, and the citation rule is repeated at the end of each request, because small models follow the most recent instruction most

reliably. Only the last two conversation turns are included, trimmed to 600 characters each. After the answer streams in, the cited numbers are parsed, numbers outside the provided range are ignored, and the interface shows only the passages actually cited.

Context window and token budget

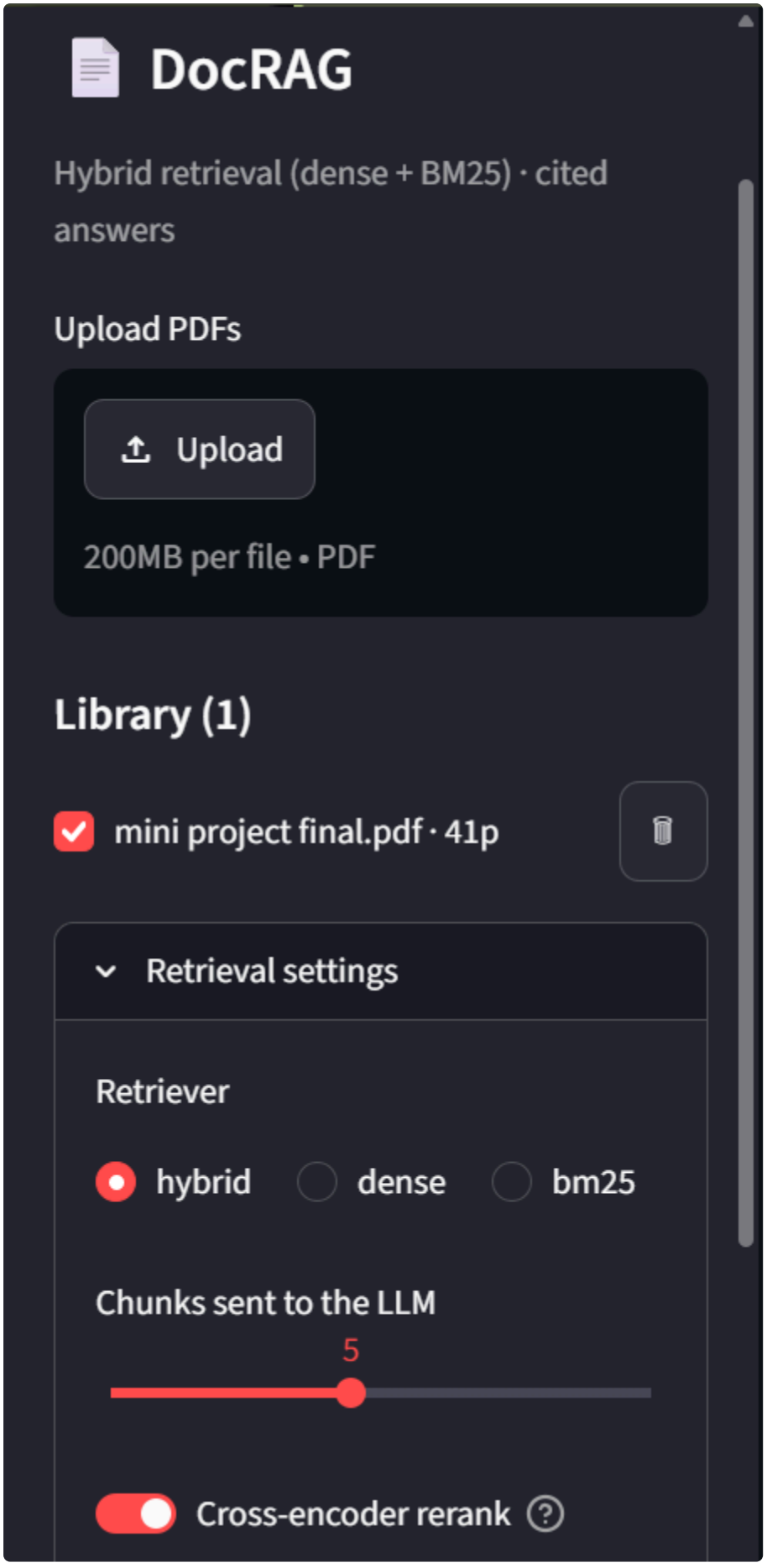
Each model can read only a limited number of tokens at once, and Ollama allocates less than the models' maximum unless asked for more. Measured with `ollama ps` while the app was answering:

Model	Maximum it supports	Window Ollama allocated	Typical use in DocRAG
`qwen2.5:3b` (answers)	about 32,000 tokens	4,096 tokens	about 2,300 to 2,700 per question
`nomic-embed-text` (embeddings)	about 8,000 tokens	2,048 tokens	about 250 per chunk

A question's budget is roughly 250 tokens of instructions, 1,300 to 1,500 for five passages, up to 600 for chat history, and 200 to 400 for the answer, which fits the 4,096-token window with some headroom. When a prompt exceeds the window, Ollama silently drops its beginning, where the instructions are. This is why history is trimmed, the number of passages is limited, and the citation rule is repeated at the end. Raising the number of passages well above 5 would need a larger, explicitly configured window.

User interface

The Streamlit app has a sidebar for documents and settings, and a chat area for questions and cited answers.


DocRAG
Hybrid retrieval (dense + BM25) · cited answers
Upload PDFs
Upload
200MB per file • PDF
Library (1)
mini project final.pdf · 41p
Retrieval settings
Retriever
hybrid
dense
bm25
Chunks sent to the LLM
5
Cross-encoder rerank

The sidebar: upload, a library with one indexed 41-page report, and the retrieval settings with hybrid search, 5 passages and reranking on.

Area	What it offers
Upload and index	Upload one or more PDFs and index them; a message reports pages and chunks per file, or that a file was already indexed
Library	Every indexed document with its page count; a checkbox per document limits which ones are searched, and a delete button removes it from the index
Retriever	Choose hybrid (default), dense only or BM25 only, mainly to compare methods
Chunks sent to the model	Slider from 1 to 10 passages, 5 by default: more gives the model more context but answers more slowly
Cross-encoder rerank	Turns the second-stage reranker on or off; on by default
Show search query	Displays the exact text searched, which makes follow-up rewriting visible
Status box	A spinner with each step as it happens (understanding a follow-up, searching, re-ranking, passages found, writing), collapsing to a summary such as "Found 5 passages · answered in 30.5s"
Answer and sources	The answer streams in word by word; below it, "Sources" lists only the cited passages with file, page and text, or "Retrieved passages" when nothing was cited
Clear chat	Starts a new conversation without touching the index
Index guard	If the index was built with a different embedding model, an explanation and a button to clear the index and start fresh

Technology stack

Layer	Technology	Role
Language	Python 3.12	All application code
Framework	None: a custom pipeline, no LangChain or LlamaIndex	Each stage is implemented directly, for full control, easy testing and measurable changes
Answer model	`qwen2.5:3b` via Ollama	Writes answers and rewrites follow-up questions
Embedding model	`nomic-embed-text` via Ollama	Turns chunks and questions into vectors
Reranker	`cross-encoder/ms-marco-MiniLM-L-6-v2` (sentence-transformers, PyTorch CPU)	Re-scores the top 20 candidates
Vector store	ChromaDB (persistent, cosine similarity)	Stores chunks, vectors and metadata
Keyword search	rank_bm25	BM25 scoring over the same chunks
PDF parsing	pypdf	Extracts text page by page
Interface	Streamlit	Chat UI with upload, library and settings
API	FastAPI, Uvicorn	REST endpoints for upload, listing, deletion and questions
Packaging	Docker, Docker Compose	Containers for Ollama, model download and the app
Testing	pytest, reportlab	Offline tests with generated PDFs
Version control	Git, GitHub	Source hosting at github.com/Taserface0/docrag

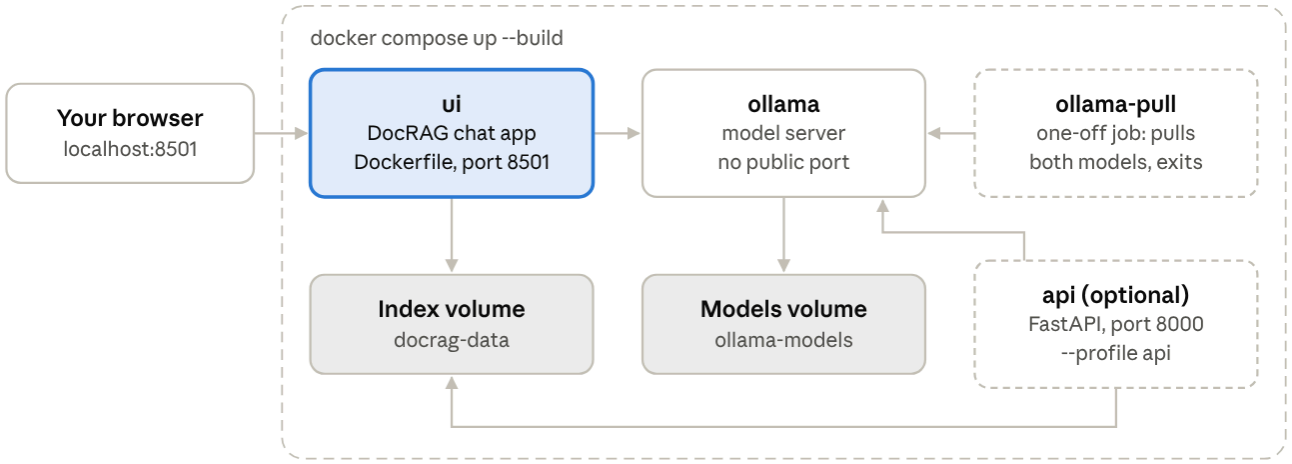
Deployment

DocRAG runs either directly with Python and a local Ollama install, or as a set of containers started with one command.

Direct run: install the Python requirements and Ollama, pull `qwen2.5:3b` and `nomic-embed-text`, copy `.env.example` to `.env`, then start the interface with `streamlit run app.py` or the API with `uvicorn api:app`. All settings (models, chunk size, number of passages, reranking) are environment variables.

Docker Compose: only Docker is needed. Ollama runs in its own container, a one-off job downloads both models on the first start, and the app starts once the models are ready. The image installs CPU-only PyTorch and pre-downloads the reranker, and `.dockerignore` keeps `.env` and local data out of it.

One command starts the model server, downloads the models and launches the app



Docker Compose setup · 4 services, 2 volumes

Dashed boxes run only some of the time: the model download finishes and exits, and the API starts only with `--profile api`. The two volumes keep about 2.5 GB of models and the document index between restarts, so only the first start is slow. The Compose files are validated but a full build has not yet been run.

How to run it

The direct route takes about 15 minutes on a first install, most of it downloading the two models (about 2.5 GB). Commands are shown for Windows PowerShell; macOS and Linux differences are noted.

1. Prerequisites

- **Python 3.12** from python.org
- **Git** from git-scm.com
- **Ollama** from ollama.com (for the direct route), or **Docker Desktop** (for the Docker route)

- About 10 GB of free disk space and at least 8 GB of RAM

2. Get the code

```
git clone https://github.com/Taserface0/docrag.git
cd docrag
```

3. Install the Python requirements

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

On macOS or Linux, activate with `source .venv/bin/activate`. The install includes PyTorch for the reranker, so it takes a few minutes.

4. Download the models

With Ollama installed and running:

```
ollama pull qwen2.5:3b
ollama pull nomic-embed-text
```

5. Configure

```
copy .env.example .env
```

On macOS or Linux, use `cp .env.example .env`. The defaults work as they are. Check that `.env` contains `OLLAMA_BASE_URL=http://127.0.0.1:11434` and `USE_RERANKER=true`.

6. Start the app

```
python -m streamlit run app.py
```

Open http://localhost:8501, upload a PDF in the sidebar, click **Index uploaded files**, and ask a question. To keep the app private to your own computer, start it with `--server.address localhost`.

7. Optional

Task	Command
Start the REST API (docs at http://localhost:8000/docs)	`uvicorn api:app --reload`
Run with Docker instead of steps 3 to 6	`docker compose up --build`, then open http://localhost:8501
Run the automated tests	`python -m pytest -q`
Build an evaluation test set from indexed PDFs	`python -m eval.generate_testset --n 30`
Measure retrieval, including reranking	`python -m eval.evaluate --k 5 --rerank`

8. Common problems

Symptom	Cause	Fix
Streamlit asks for an email on first start	Optional newsletter prompt	Press Enter, or start with `--server.headless true`
Every search or answer is about 2 seconds slower than expected	On Windows, `localhost` tries IPv6 first	Use `http://127.0.0.1:11434` for `OLLAMA_BASE_URL`
"Could not connect to Ollama"	Ollama is not running	Start the Ollama app, then try again
Error mentioning `ollama pull`	A model has not been downloaded	Run the `ollama pull` command shown in the message
"The index was built with…" message	The embedding model or prefixes changed since indexing	Click **Clear the index and start fresh**, then re-index
"Reranker not installed"	`sentence-transformers` is missing	`pip install sentence-transformers`, or turn reranking off
"No extractable text"	The PDF is a scanned image	Run OCR on it first, for example with `ocrmypdf`
Port 8501 already in use	An earlier Streamlit is still running	Close that terminal; Streamlit otherwise switches to port 8502

Configuration and API reference

Settings

All settings are environment variables, usually set in `.env`. Restart the app after changing them.

Setting	Default	What it controls
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Where the Ollama server is; Docker sets it to the `ollama` container

Setting	Default	What it controls
`CHAT_MODEL`	`qwen2.5:3b`	Model that writes answers and rewrites follow-ups
`EMBED_MODEL`	`nomic-embed-text`	Model that makes the vectors; changing it requires re-indexing
`EMBED_BATCH_SIZE`	`32`	Chunks sent per embedding request
`EMBED_DOC_PREFIX`, `EMBED_QUERY_PREFIX`	the model's recommended prefixes	Task prefixes for passages and questions
`DATA_DIR`	`./data`	Folder for the index
`COLLECTION`	`docrag`	Name of the index collection
`CHUNK_SIZE`	`1000`	Target chunk length in characters
`CHUNK_OVERLAP`	`150`	Characters shared by neighbouring chunks
`CANDIDATES`	`20`	Passages each search method returns before fusion
`TOP_K`	`5`	Passages sent to the model
`USE_RERANKER`	`true`	Whether the cross-encoder reranks the fused candidates
`RERANK_CANDIDATES`	`20`	How many fused candidates the reranker scores; `0` means all
`RERANKER_MODEL`	`cross-encoder/ms-marco-MiniLM-L-6-v2`	Reranking model

REST API

Start with `uvicorn api:app`; interactive documentation is served at `/docs`.

Method and path	Input	Returns
`GET /health`	none	Status and the number of indexed chunks
`POST /documents`	A PDF as multipart field `file`	Document ID, file name, pages, chunks, and whether it was already indexed
`GET /documents`	none	Every indexed document with its ID, name, pages and chunk count
`DELETE /documents/{doc_id}`	Document ID in the path	Confirmation, or 404 if unknown
`POST /ask`	JSON: `question`, optional `history`, `doc_ids`, `top_k`, `mode` (hybrid, dense or bm25) and `rerank`	The answer, the search query used, and each source with number, file, page, text, score and whether it was cited

Example from macOS, Linux or Git Bash (in Windows PowerShell, the interactive /docs page is easier):

```
curl -F file=@report.pdf http://localhost:8000/documents
curl -X POST http://localhost:8000/ask -H "content-type: application/json" -d
'{"question": "What frontend framework is used?"}'
```

Evaluation

Hybrid search with reranking found the right passage in the top 5 for 27 of 30 questions (90%), against 22 of 30 (73%) for dense search alone.

Method

The test set was generated automatically from the indexed document, a 41-page project report. Thirty chunks were sampled at random, and for each one the local model wrote a question that the chunk answers. Because each question comes from a known chunk, retrieval can be scored without hand labelling. Each question was then run through every retrieval configuration, and four measures were recorded:

- **hit@1:** the source chunk ranked first.
- **hit@5:** the source chunk was among the five passages the model reads. This is the most important measure.

- **MRR (mean reciprocal rank):** the average of 1 / rank of the source chunk, so 1.0 means always first.
- **page@5:** any of the top five came from the right page, a lenient check.

Results

Retriever	hit@1	hit@5	MRR	page@5	Search time per question
Dense only	0.467	0.733	0.567	0.900	65 ms
BM25 only	0.533	0.767	0.631	0.800	0.7 ms
Hybrid (RRF)	0.533	0.800	0.631	0.900	60 ms
Hybrid + rerank, all candidates	0.533	0.900	0.696	0.967	1,889 ms
Hybrid + rerank, top 20 (default)	**0.533**	**0.900**	**0.696**	**0.967**	**1,266 ms**

Laptop CPU, k = 5, 30 questions.

Findings

- **Hybrid beats either method alone.** BM25 did well because the report is full of exact names (SmartRecco, ChromaDB, Django); combining both catches what each one misses.
- **Reranking gave the largest gain,** from 80% to 90% hit@5, mostly by pulling relevant passages from just outside the top five into it. It did not change which passage came first.
- **Reranking only the top 20 fused candidates kept identical accuracy and was 33% faster** than reranking all of them, so it is the default.
- **Caveat:** with 30 questions, one question is worth about 3 percentage points, so small gaps are indicative rather than conclusive. Questions written from a single chunk also slightly favour keyword overlap. A larger, partly hand-written test set would make the comparison more reliable.

Performance optimisations and qualitative testing

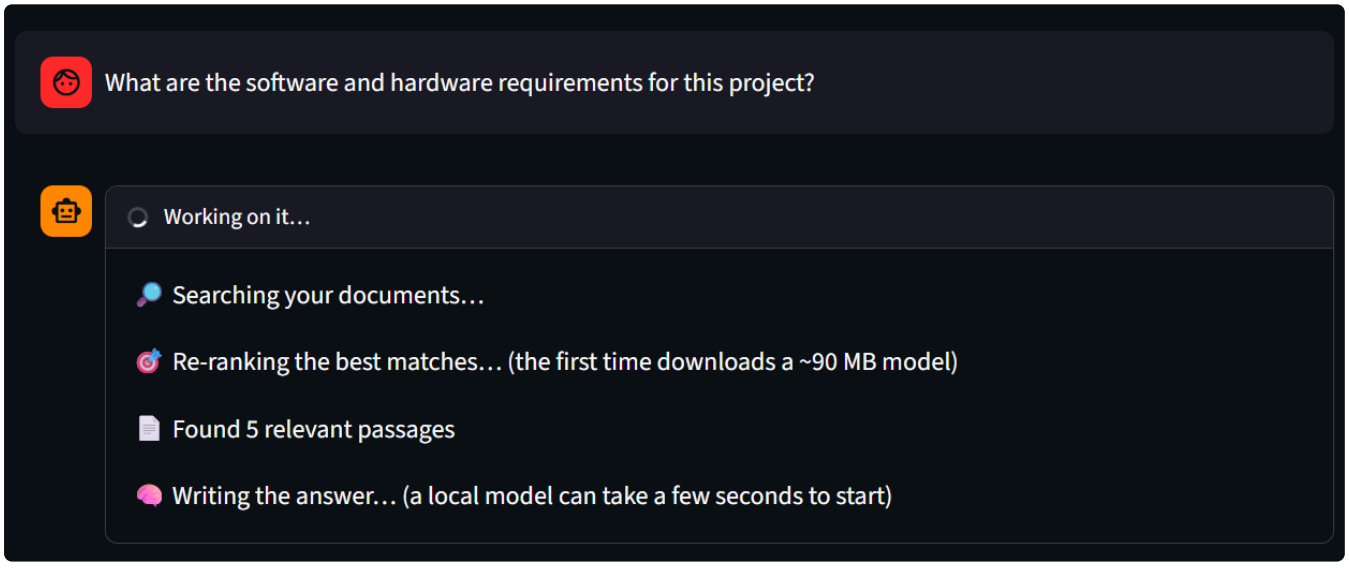
The largest speed-up came from a one-line fix found through the evaluation's timing column: dense search dropped from 2,117 ms to 59 ms per question.

Optimisations

Change	Problem it solved	Effect
Ollama address `127.0.0.1` instead of `localhost`	On Windows, `localhost` tried IPv6 first and added about 2 s to every model call	Dense search 2,117 ms to 59 ms (about 35 times faster); faster answers too
Batched embeddings via `/api/embed`	One HTTP request per chunk made indexing slow	A 100-page PDF needs about 13 requests instead of about 400
Rerank the top 20 fused candidates only	Reranking every candidate was slow on CPU	1,889 ms to 1,266 ms per question, same accuracy
Rewrite only real follow-ups	Every question after the first cost an extra model call	A self-contained question took 30.5 s instead of 45 s
Trimmed chat history	Long prompts are slow on CPU and risk the context window	At most 2 turns of 600 characters each

Live progress indicator

On a CPU, the model can take 30 seconds or more before its first word. The interface shows a status box with a spinner and each step as it happens: understanding a follow-up, searching, re-ranking, the number of passages found, and writing the answer. It stays until the first words arrive, then collapses to a summary such as "Found 5 passages · answered in 30.5s".



The status box mid-answer: search and reranking are done, five passages were found, and the model is starting to write.

Qualitative testing

The system was tested by hand with questions about the indexed report, covering a simple fact, an explanation, a follow-up, a multi-part question and a question whose answer is not in the document.

Behaviour	First prompt version	After prompt and pipeline changes
Answers with citations	3 of 6	All answerable questions
Headings in answers	Large headings in a list answer	None
Question not in the document	Correctly refused	Correctly refused, 30.5 s instead of 45 s
Follow-up "Why is that better…"	Answered, without citations	Briefly refused after a history trim; fixed by giving the model the rewritten question, now answered with a citation

Example exchange

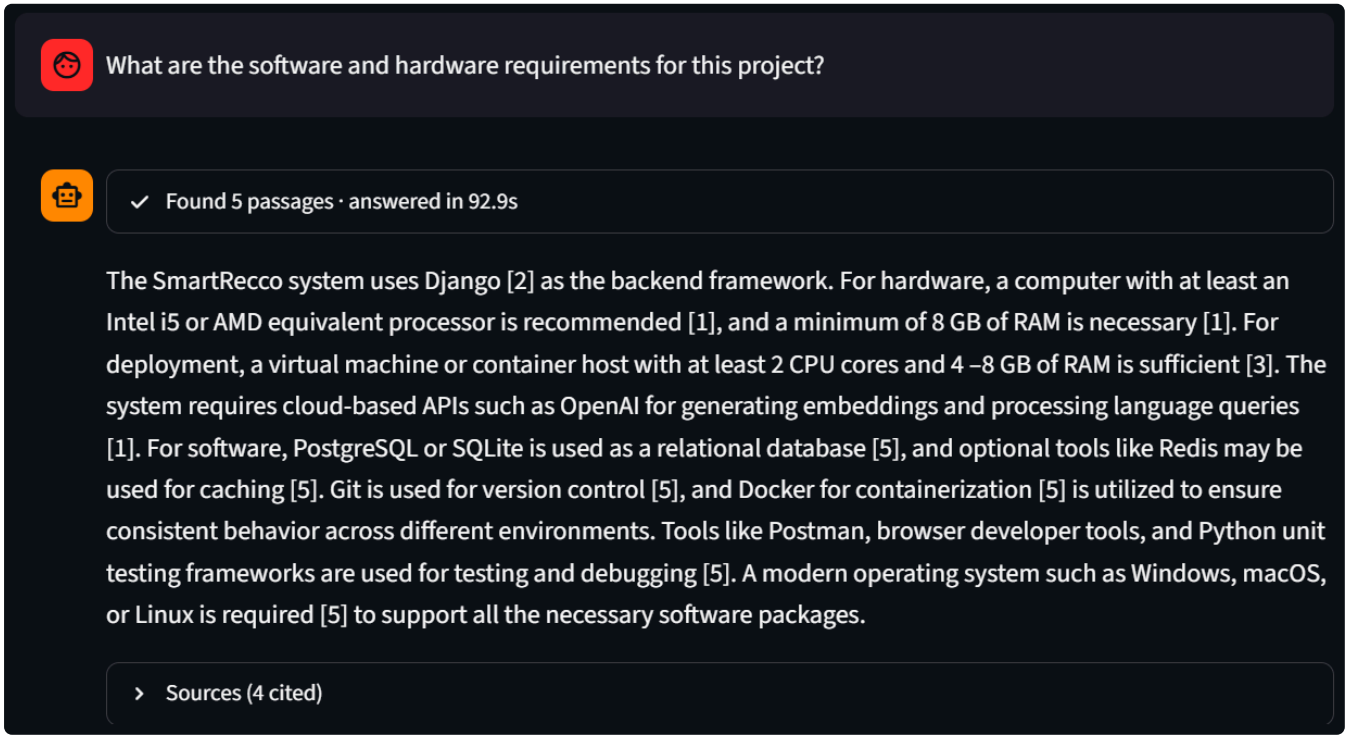
A question followed by a follow-up, from the final test run against the indexed report:

> **Question:** How does SmartRecco use ChromaDB to improve recommendations?
>
> **Answer:** [1] SmartRecco uses ChromaDB to improve recommendations by enabling efficient storage and querying of vector representations. This allows the system to retrieve items based on conceptual or contextual closeness, making recommendations more intuitive and aligned with the user's emotional and contextual needs. *(Sources: 1 cited)*
>
> **Follow-up:** Why is that better than traditional recommendation systems?
>
> **Answer:** [1] ChromaDB is better than traditional recommendation systems for SmartRecco because it enables efficient storage and querying of vector representations, allowing the system to retrieve items based on conceptual or contextual closeness. This design makes SmartRecco more intuitive by understanding the meaning behind user queries rather than just the words they use. *(Sources: 1 cited, answered in 45.0 s)*

The follow-up contains "that", so it was rewritten into a standalone question before searching, and the model correctly resolved it to ChromaDB and semantic search. Asked "What was the project's total budget in rupees?", which the report does not cover, the

system replied "I couldn't find that in the uploaded documents."

A broader question draws on several parts of the report. With reranking on, the answer below cites four of the five passages, one after each fact:



This answer took 92.9 s because it was the first question after starting the app, when the reranker model is loaded; later questions are faster.

The remaining weaknesses come from the 3-billion-parameter model rather than the pipeline: on narrow questions it often cites a single passage and places the citation at the start of the answer, and on follow-ups it can repeat a previous answer instead of using a sharper passage. With reranking on, broad questions draw on several passages with a citation after each fact, as in the requirements answer above.

Security and privacy

Document content never leaves the machine: parsing, embedding, storage, reranking and answer generation all run locally, and the models are not trained on the documents. An earlier version used a hosted API, which sent every chunk and question to an outside service; moving to Ollama removed that.

Area	Current state	Recommended for a production deployment
Where models run	Local, through Ollama	Self-hosted or private-cloud models for confidential data

Area	Current state	Recommended for a production deployment
Data sent to the model	Only the 5 retrieved passages	Unchanged, plus redaction of personal data at ingestion
Access to the interface	No login; reachable by anyone on the same network unless started with `--server.address localhost`	Authentication and single sign-on
Who can see which document	A document filter exists, but there are no users	Permission-aware retrieval: filter chunks by the user's access rights before they reach the model
Data at rest	Chunks stored unencrypted in `data/`	Encrypted storage and managed keys
Secrets and repository	`.env` and `data/` excluded from Git and from Docker images	Unchanged
Malicious documents	A PDF could contain instructions aimed at the model (prompt injection); no tools or internet access limit the impact to wrong answers	Content filtering and guardrails
Auditing	None	Logs of who asked what and which passages were retrieved

Automated testing

The project has 25 automated tests, run with `pytest`, and all pass. They run fully offline: a hashing embedder and a fake language model stand in for Ollama, and test PDFs are generated on the fly, so the suite needs no models, network or GPU.

Area	What is checked
Chunking	Chunk size limits and overlap between neighbouring chunks
Ingestion	Re-uploading the same file is skipped; documents are listed and deleted correctly

Area	What is checked
Retrieval	BM25 finds exact codes; hybrid search ranks the right page first; filtering by document; Reciprocal Rank Fusion rewards agreement
Reranking	The reranker reorders candidates; only the configured number of candidates is scored; reranking is on by default
Embeddings	Batched requests, document and query prefixes, fallback for older Ollama versions, clear error when a model is missing
Index safety	An index built with a different embedding model is refused; an empty legacy index is rebuilt
Conversation	Follow-ups are detected and rewritten; self-contained questions skip the rewrite; the model receives the rewritten question
Prompt and citations	The citation rule is repeated at the end of each request; history is trimmed; out-of-range citation numbers are ignored
API and evaluation	Upload, list, ask and delete endpoints; evaluation metrics are computed correctly
Streaming	Answers stream token by token

The Streamlit interface was additionally checked with Streamlit's test runner, including the progress steps and the clear-index flow.

Limitations, future work and conclusion

Limitations

- **Speed on CPU:** answers take about 30 seconds, 45 for follow-ups, because the model runs without a GPU.
- **Small model:** the 3-billion-parameter model often cites a single passage on narrow questions and sometimes misses a sharper passage.
- **Scanned PDFs:** files without extractable text are rejected; they need OCR first.
- **Keyword index in memory:** BM25 is rebuilt after changes, which suits hundreds of documents rather than millions.
- **Small test set:** 30 synthetic questions make small differences indicative only.

- **Docker setup untested:** the Compose files are written and validated, but a full build has not yet been run.
- **No users or login:** anyone who can reach the interface can see and query every document.

Future work

- Test a larger model (for example `qwen2.5:7b`) and GPU acceleration for faster, better-cited answers.
- Set an explicit context window (for example 8,192 tokens) instead of relying on Ollama's 4,096-token default.
- Grow the test set to about 100 questions, including hand-written ones, and add answer-quality grading.
- Add a provider setting to switch between local models and a hosted API for public documents.
- Add authentication, per-user document permissions, encryption at rest and audit logs.
- Support scanned PDFs through OCR, and table extraction.

Conclusion

DocRAG shows that a compact, framework-free pipeline can answer questions about private documents with verifiable citations while running entirely on a laptop. Combining keyword and semantic search, then reranking, lifted top-5 retrieval accuracy from 73% to 90%. Measuring each change, rather than assuming it helped, guided every design decision and exposed a 35-fold latency problem that manual testing had missed.

References

The methods DocRAG implements come from these works.

1. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401
2. Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval.
3. Cormack, G. V., Clarke, C. L. A. and Büttcher, S. (2009). Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009.
4. Nogueira, R. and Cho, K. (2019). Passage Re-ranking with BERT. arXiv:1901.04085
5. Bajaj, P. et al. (2016). MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. arXiv:1611.09268

6. Nussbaum, Z. et al. (2024). Nomic Embed: Training a Reproducible Long Context Text Embedder. arXiv:2402.01613
7. Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115
8. Source code: github.com/Taserface0/docrag